

The UM6P Vanguard Center

AI Tutor Engineering Internship

ENGINEERING WORK PACKAGE Final

Getting Started with Ontologies

AI-TUTOR-WP-003

Owner	Islam Bouikiri
Mentor	Jalal Maaouni, Asmae Afifi
Start	2026-07-31
Target	2026-08-10
Review cadence	Daily checkpoint
Repository	Your private Github Repo
Submission	Submit the artifact on Github.

Executive brief

Study the basic terms and tools of RDF, OWL, and knowledge graphs. Inspect the supplied OWL ontology. Query the ontology with Protégé, RDFLib, and SPARQL. Build a small LangChain agent that uses deterministic graph tools. Use the ontology as the only source of truth.

1 Objective

Objective

Build an OWL knowledge graph explorer for `comp101_L10.owl`. First, inspect the ontology without an LLM. Then, implement deterministic query functions. After the deterministic functions pass their tests, expose selected functions as LangChain tools. Make the agent answer from graph evidence only.

2 Source material

Use the supplied file:

```
1 ontology/comp101_L10.owl
```

Treat the OWL file as read-only input. Do not edit the source file. Do not replace the source file with an LLM summary. Do not convert the source file to a simpler format before the first inspection.

3 Reading and research task

Complete the reading task before you implement the agent. Use official specifications and official tool documentation as the main sources.

Theme	Material	Priority
RDF data model	W3C RDF 1.1 Concepts and Abstract Syntax. Study RDF graphs, triples, IRIs, blank nodes, and literals.	Required
OWL ontology model	W3C OWL 2 Primer. Study classes, individuals, properties, axioms, and reasoning.	Required
SPARQL queries	W3C SPARQL 1.1 Query Language. Study SELECT , ASK , FILTER , and property paths.	Required
RDFLib	RDFLib Getting Started. Study graph parsing, namespace use, triple iteration, and SPARQL execution.	Required
Protégé	Protégé Getting Started. Study the Entities view, the hierarchy views, search, and the reasoner.	Required
OWL reference	W3C OWL 2 Quick Reference. Use this source when the primer does not give enough detail.	Reference

3.1 Research topics

Research and explain these terms:

- RDF graph, triple, subject, predicate, and object.
- IRI, namespace, prefix, literal, and blank node.
- Ontology, knowledge graph, schema, and instance data.
- OWL class and named individual.
- Object property and datatype property.
- Domain and range.
- Inverse, symmetric, asymmetric, and transitive properties.
- Disjoint classes.
- Asserted fact and inferred fact.
- Open-world assumption.
- SPARQL query and graph traversal.

3.2 Reading artifact

Create `kg_reading_notes.md`. Use your own words. Add a source link for each section.

Use this structure:

1. Write an executive synthesis of 400 to 600 words.
2. Compare RDF, RDFS, OWL, and SPARQL in one table.

3. Give five examples of asserted facts.
4. Give five examples of inferred facts.
5. Explain the open-world assumption with two examples.
6. List five questions that the ontology can answer.
7. List five questions that the ontology cannot answer.
8. Record all open questions for the review meeting.

Do not copy definitions. Do not use generated text. Be ready to explain each term during the review.

4 Tool setup

Install and use these tools:

- Protégé Desktop.
- RDFLib.
- Pytest.
- Ollama.
- LangChain.
- `langchain-ollama`.

Create a clean virtual environment. Pin the package versions in `requirements.txt`.

5 Work sequence

Follow the sequence below. Do not start the next step before the current step works.

5.1 Step 1: Inspect the ontology in Protégé

Open `comp101_L10.owl` in Protégé.

Record the following information:

- Ontology IRI and version.
- Namespace prefixes.
- Named classes.
- Object properties.
- Datatype properties.
- Named individuals.
- Property characteristics.
- Domain and range declarations.
- Inverse property declarations.

- Disjoint class declarations.

Run the HerMiT reasoner. Compare asserted information with inferred information. Record at least five differences. If the reasoner reports an inconsistency, stop. Record the error. Report the error to the mentor.

5.2 Step 2: Inspect the ontology with RDFLib

Create `inspect_ontology.py`. Load the RDF/XML file with RDFLib.

```

1 from pathlib import Path
2 from rdflib import Graph
3
4 ONTOLOGY_PATH = Path("ontology/comp101_L10.owl")
5
6 graph = Graph()
7 graph.parse(ONTOLOGY_PATH, format="xml")
8
9 print(f"Triples: {len(graph)}")

```

Print a stable inventory. Include counts and labels. Do not print raw IRIs only. Show the local identifier and the human-readable label.

5.3 Step 3: Write SPARQL queries

Create at least 10 SPARQL queries. Store the queries in `queries.sparql`. Add a short comment before each query.

Include queries for these tasks:

- List all named classes.
- List all named individuals and their types.
- Find an entity from its `rdfs:label`.
- Find an entity from its `courseId`.
- List all relations of one entity.
- Find direct dependencies.
- Find transitive dependencies with a property path.
- Find entities that contrast with a selected entity.
- Find methods and their produced types.
- Find methods and their possible errors.

Use prefixes. Do not repeat full IRIs in each query.

5.4 Step 4: Build the deterministic graph service

Create `graph_service.py`. Implement deterministic functions. Do not use the LLM in this file.

Implement at least these functions:

```

1 def list_classes() -> list[dict]:
2     ...
3
4 def list_entities_by_type(type_label: str) -> list[dict]:

```

```

5     ...
6
7 def find_entity(label_or_id: str) -> list[dict]:
8     ...
9
10 def describe_entity(label_or_id: str) -> dict:
11     ...
12
13 def get_relations(label_or_id: str) -> list[dict]:
14     ...
15
16 def get_dependencies(label_or_id: str, transitive: bool = False) -> list[dict]:
17     ...
18
19 def find_path(source: str, target: str, max_depth: int = 5) -> dict:
20     ...

```

Return structured data. Use dictionaries and lists. Include the IRI, the course identifier, the label, and the relation. Do not return prose as the only result.

Handle these conditions:

- No entity matches the input.
- More than one entity matches the input.
- The entity has no value for the requested relation.
- The path does not exist within the depth limit.
- A label differs only by case.

5.5 Step 5: Add LangChain tools

Expose selected graph service functions as LangChain tools. Keep the tools thin. Do not copy query logic into `tools.py`.

```

1 from langchain_core.tools import tool
2
3 @tool
4 def describe_ontology_entity(label_or_id: str) -> dict:
5     """Return authoritative ontology data for one entity."""
6     ...
7
8 @tool
9 def find_concept_dependencies(
10     label_or_id: str,
11     include_transitive: bool = False,
12 ) -> dict:
13     """Return direct or transitive dependencies from the ontology."""
14     ...
15
16 @tool
17 def find_relation_path(source: str, target: str) -> dict:
18     """Return one relation path between two ontology entities."""
19     ...

```

Use ChatOllama and `create_agent`. Use temperature 0 for the first tests.

The system prompt must include these rules:

- Use a graph tool before each factual answer about the lecture.
- Use only facts that the graph tool returns.
- State when the ontology does not contain the requested information.
- State when an entity name is ambiguous.
- Include the course identifier or IRI in the answer.
- Do not use general model knowledge to complete missing facts.

6 Questions that the agent must handle

Test questions such as these:

- What is Encapsulation?
- What does Encapsulation depend on?
- What concepts depend on Class?
- What contrasts with Inheritance?
- What enables Instantiation?
- What type does `__repr__` produce?
- Which error can `__repr__` throw?
- Show a path from Class to Data Hiding.
- What is gradient descent?
- What does it depend on?

The last two questions test refusal and clarification. Do not reward a fluent answer when the ontology has no evidence.

7 Acceptance criteria

Acceptance criteria

The work is accepted when all criteria below are true.

- ✓ Protégé loads the ontology without a parse error.
- ✓ The intern can explain the required RDF and OWL terms without notes.
- ✓ RDFLib loads the ontology and reports a stable inventory.
- ✓ At least 10 SPARQL queries return the expected results.
- ✓ The graph service has deterministic unit tests.
- ✓ The graph service returns structured results.
- ✓ The agent calls a graph tool before a factual ontology answer.
- ✓ The agent includes an entity identifier in each factual answer.
- ✓ The agent reports unknown and ambiguous entities.

- ✓ The agent does not answer unsupported questions from model memory.
- ✓ The evaluation set contains at least 30 cases.
- ✓ The report includes failures and unresolved questions.